

1. Problem Framing

Attaching every available tool’s schema directly into an LLM’s context does not scale. Two failure modes show up as the tool count grows past a few dozen:

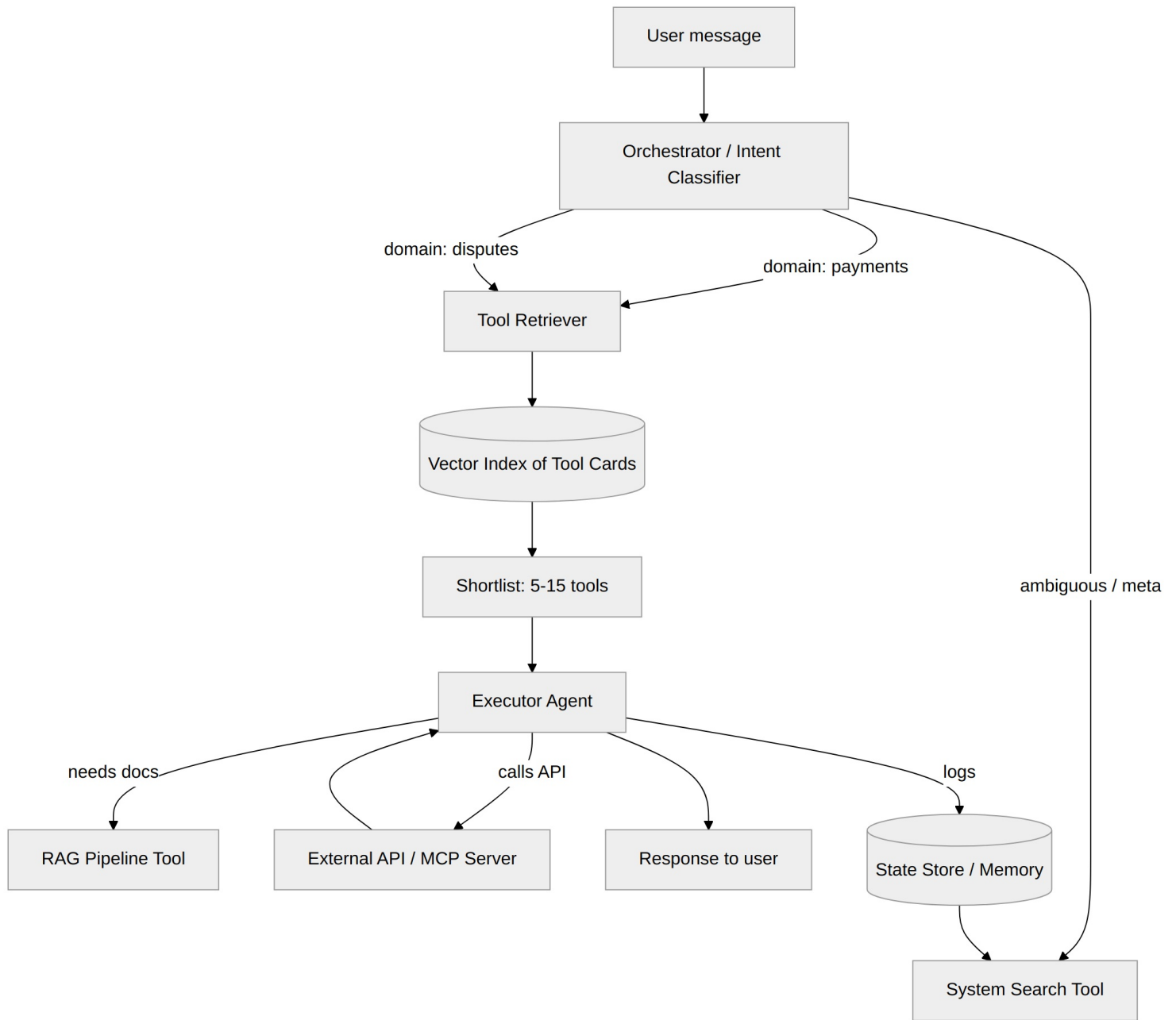
- **Selection accuracy collapses.** With 50–500+ near-duplicate schemas (e.g., `create_invoice`, `send_invoice`, `create_payment`, `create_payout`), the model’s attention is spread thin across semantically overlapping options, and it increasingly picks the wrong tool or the wrong overload of a similar tool.
- **Parameter hallucination increases.** Long tool-definition blocks push the actual user intent further from the point of generation, and the model starts inventing plausible-looking arguments instead of grounding them in the schema.
- **Cost and latency balloon.** Every turn re-sends every schema, which is wasteful token-wise even before accuracy degrades.

The design goal is therefore not “let the model see every tool” but **“give the model the smallest, most relevant slice of the tool universe at each step, dynamically, while preserving the ability to reach any of the thousands of tools when actually needed.”** This is fundamentally a *retrieval and routing* problem layered on top of an *execution* problem, not a single-prompt problem.

2. High-Level Architecture

The system is organized as a **hierarchical, retrieval-gated agent** with four layers:

1. **Tool Catalog & Index** — an offline-built, continuously-updated registry of every tool (from Postman collections, OpenAPI specs, internal RAG/search tools) with rich metadata and embeddings.
2. **Orchestrator (Planner) Agent** — a lightweight LLM call that classifies user intent into a *domain/capability*, not a specific tool.
3. **Tool Retriever + Sub-Agent Router** — narrows thousands of tools down to a shortlist (typically 5–15) using semantic search, metadata filters, and usage priors, then hands that shortlist to a specialized executor.
4. **Executor Agent(s)** — the actual tool-calling LLM, which only ever sees the shortlisted tools, plus the RAG tool and the System Search tool as always-available utilities.



This mirrors a pattern several production systems have converged on independently: treat tool selection as a **RAG problem over tool descriptions** (“tool RAG”), then delegate execution to a narrowly-scoped agent, optionally organized as a **multi-agent hierarchy** where each sub-agent owns one API domain (payments, disputes, reporting, etc.) — similar in spirit to how LangGraph’s supervisor pattern or CrewAI’s crew-of-specialists pattern organize work.

3. Tool Catalog & Indexing Layer

Every tool — a PayPal “create invoice” endpoint, the RAG tool, the System Search tool, or any of the other 500 APIs — is normalized into a **Tool Card**:

```

{
  "tool_id": "paypal.invoices.create",
  "name": "create_invoice",
  "domain": "payments.invoicing",
  "short_desc": "Create and optionally send a PayPal invoice to a recipient.",
  "long_desc": "Use when the user wants to bill someone a specific amount...",
  "parameters_schema": { "...": "JSON schema" },
  "examples": ["Send an invoice for $50 to jane@example.com"],
  "tags": ["invoice", "billing", "create"],
  "auth_scope": "invoicing:write",
  "cost_class": "write",
  "avg_latency_ms": 480,
  "embedding": [0.021, -0.014, "..."]
}

```

Tool Cards are:

- **Generated automatically** from OpenAPI/Postman specs where possible, with an LLM pass to write the `short_desc/long_desc/examples` fields — this

is what lets the design scale from 50 to 5,000 tools without hand-authoring every entry.

- **Embedded** (e.g., via a text-embedding model) and stored in a vector database (Qdrant, pgvector, Weaviate) so they can be retrieved by semantic similarity to the user’s request, not just keyword match.
- **Indexed with structured metadata** (domain, auth scope, read vs. write, cost/risk class) so retrieval can be filtered, not only ranked — this matters a lot for safety (see Section 7): a “read sales report” request should never even surface destructive write tools as candidates.
- **Versioned**, so that when an upstream API changes, the Tool Card and its embedding are regenerated rather than silently going stale.

This catalog is the single source of truth. Both the Orchestrator and the Executor query it; neither ever holds a static list of “all tools” in its prompt.

4. Tool Selection & Routing

Routing happens in two stages, which is the core mechanism that keeps accuracy from degrading as the catalog grows:

Stage 1 — Coarse domain routing (Orchestrator). A small, cheap LLM call (or even a classifier/embedding-similarity lookup, no generation needed) maps the user message to one or more high-level domains: `payments.invoicing`, `payments.disputes`, `reporting`, `rag`, `system_meta`. This step sees only domain names, not tool schemas, so it stays accurate even with thousands of underlying tools.

Stage 2 — Fine-grained tool retrieval (Retriever). Within the chosen domain(s), the system embeds the user request and does a top-k similarity search against the Tool Card index ($k \approx 5\text{--}15$), optionally re-ranked with a cross-encoder for precision, and optionally filtered by conversation state (e.g., “user already authenticated for scope X”). Only this shortlist — full schemas included — is placed into the Executor’s context.

This is the same idea behind “tool RAG”/dynamic tool-loading approaches used by frameworks such as LangGraph’s `create_react_agent` with a retrieval-backed tool node, or Anthropic’s own guidance on tool search for large tool sets: don’t put static tool definitions in-context, look them up per turn. It directly targets the root cause in Section 1 — the model never has to discriminate among 500 options, only among ~ 10 .

Multi-step / compound requests (e.g., “check if there’s an open dispute for user_123, and if so refund them”) are handled by letting the Executor itself call the Retriever again mid-trajectory as a tool (“search_tools(query)”), so the shortlist can expand across a single conversation without ever being fully loaded at once. This is essentially the **ReAct loop** (reason → act → observe → repeat) with a retrieval action folded in as a first-class step.

5. State Management

State is split into three tiers so that the Executor’s context stays small even in long sessions:

- **Working memory (per-turn):** the current shortlist of tools, the last few tool call/response pairs, summarized rather than kept verbatim once older than N turns.
- **Session state (per-conversation):** structured facts extracted along the way — recipient emails already confirmed, currency, the user’s account ID, in-flight multi-step plans (e.g., “step 2 of 3: awaiting dispute confirmation before issuing refund”) — kept as a small JSON object, not as raw transcript, and passed explicitly to whichever sub-agent is active. This is the pattern LangGraph calls a shared/typed graph state, and it’s what lets the Orchestrator hand off to a different domain sub-agent mid-conversation without losing context.
- **Durable state (cross-session):** logs of every tool call (inputs, outputs, status, latency, cost) written to a database, which doubles as the backing store for the **System Search Tool** (“what’s the status of my last request?”) and as training data to improve the retrieval layer over time (see Section 8).

6. The Two Required Utility Tools

RAG Pipeline Tool. Implemented as its own tool with its own schema (`query: string`, `top_k: int`, `filters: {...}`) rather than being special-cased. It sits behind the same retrieval layer as any API tool, so the Retriever can decide *when* the RAG tool itself is the right choice (e.g., “how do PayPal disputes work?” routes to RAG, not to an API call). Internally: chunk + embed the knowledge base, retrieve top-k chunks, optionally re-rank, and return citations alongside the answer so the Executor can ground its response.

System Search Tool. This is a search interface over the system’s own metadata: the live Tool Catalog (answers “what tools exist for managing invoices?”) and the durable state/log store (answers “what’s the status of my last request?”). Practically, it’s two sub-functions behind one tool schema — `search_capabilities(query)` and `search_logs(user_id, query)` — kept as one tool so the Executor doesn’t need to disambiguate at the top level; the tool implementation routes internally.

Both tools are always included in the Executor’s context regardless of domain, since they’re low-cost, low-risk, and needed across every domain.

7. Error Handling & Safety

- **Schema validation before execution.** Every generated tool call is validated against its JSON schema before hitting the real API; on failure, the error is fed back to the Executor as an observation (“missing required field `amount`”) rather than surfaced raw to the user, letting the model self-correct within the ReAct loop (bounded to 2–3 retries).
- **Confirmation gating on destructive actions.** Tool Cards carry a `cost_class` (`read` / `write` / `irreversible`). Anything marked `irreversible` (issuing a refund, sending a payment) requires an explicit user confirmation step before execution, surfaced back through the chat rather than auto-executed.

- **Least-privilege retrieval.** Because retrieval is metadata-filtered by the user’s actual auth scope, tools the user isn’t authorized for never even enter the shortlist — this is both a UX improvement (fewer wrong suggestions) and a safety boundary.
- **Fallback on empty/low-confidence retrieval.** If the Retriever’s top result is below a similarity threshold, the Executor is instructed to ask a clarifying question or fall back to the System Search / RAG tool rather than guessing a tool call.
- **Circuit breaking on upstream failures.** Repeated failures from a given external API within a time window mark that tool as temporarily degraded in the catalog, so the Retriever deprioritizes it and the Executor can tell the user the service is currently unavailable instead of retrying blindly.
- **Full observability.** Every routing decision (which domain, which shortlist, which tool chosen, why) and every tool call/response is logged with a trace ID, feeding an observability tool such as LangSmith (or Langfuse/Arize Phoenix as open-source alternatives) so failures can be traced turn-by-turn instead of only observed as a bad final answer.

8. Scalability: 50 Tools to 5,000 Tools

The architecture is designed so scaling the catalog is an **indexing/data problem**, not an architecture change:

- Adding a new API only means generating Tool Cards and re-embedding — no prompt changes to the Orchestrator or Executor.
- The two-stage routing (domain → shortlist) keeps the Executor’s context size roughly constant regardless of catalog size, since it only ever sees ~10–15 tools plus the two utility tools.
- Domains can be sharded across dedicated sub-agents (a payments specialist, a reporting specialist, a disputes specialist) so that as the number of domains grows, work parallelizes across sub-agents rather than one agent’s context growing.
- Retrieval quality itself can be continuously improved using the durable call logs as feedback: mis-routed calls (wrong tool selected, corrected by the user or by a retry) become training signal to fine-tune the embedding model or re-rank thresholds — a flywheel that gets better precisely as usage scales up.
- Cost scales sub-linearly with catalog size: the expensive step (full LLM tool-calling generation) always operates on a small shortlist; the cheap step (embedding similarity search) is what scales with catalog size, and vector search is efficient even at hundreds of thousands of entries.

9. Framework Choice & Trade-offs

Chosen approach: LangGraph, with a custom retrieval-gated tool-loading layer, not a stock “give the agent all tools” setup.

Option	Why considered	Why chosen / not chosen
LangGraph	Explicit, typed graph state; first-class support for multi-agent handoff (supervisor/sub-agent patterns); fine-grained control over the ReAct loop, retries, and human-in-the-loop confirmation steps	Chosen. The state-machine model maps directly onto the domain-routing + shortlist-executor design in Section 2–4, and its checkpointing gives durable state (Section 5) close to for free. Trade-off: more boilerplate and a steeper learning curve than a single-call agent library.
LangChain (agent executor alone)	Mature tool-calling abstractions, huge integration ecosystem	Good for the Executor sub-agent itself (its tool-calling and output-parsing utilities are reused inside LangGraph nodes), but its default agent loop assumes a mostly-static tool list, which is exactly the anti-pattern we’re avoiding at scale.
CrewAI	Very ergonomic for defining role-based multi-agent crews (e.g., “invoicing specialist,” “dispute specialist”)	Considered for the sub-agent layer; less flexible than LangGraph for custom state/routing logic and dynamic tool retrieval, and less mature tracing/production tooling. A reasonable alternative if the team prioritizes speed of prototyping over fine control.
LlamaIndex	Excellent for the RAG Pipeline Tool specifically (retrieval, chunking, re-ranking primitives)	Used, but only inside the RAG Pipeline Tool implementation , not as the overall agent framework — it’s a retrieval library first, agent framework second.
DSPy	Strong for systematically optimizing prompts/routing classifiers against a metric	Worth adopting later to optimize the Stage-1 domain classifier and re-ranking thresholds once real usage data exists; overkill as the initial orchestration layer.
From scratch	Full control	Rejected as the primary framework — reinventing state persistence, retries, and multi-agent handoff has little payoff versus LangGraph’s existing primitives, though the Tool Catalog/Retriever itself is custom-built regardless of framework since no framework ships a production-grade tool-RAG layer out of the box.

Observability: LangSmith is used for tracing, given its tight LangGraph integration (per-node traces, token/cost accounting, dataset-based evals for the routing classifier). Langfuse is the natural open-source fallback.

10. Summary

The central design decision is to stop treating “which tool to call” as something the final LLM call decides in one shot over the full catalog, and instead treat it as a **retrieval problem** (find the ~10 relevant tools) followed by a **narrow execution problem** (call one of those 10 correctly). Domain-based routing, a semantically-indexed Tool Catalog, tiered state management, and safety gating on destructive actions are what let this hold up whether the catalog is 50 PayPal endpoints or 5,000 tools across many services — the Executor’s cognitive load stays constant even as the system’s total capability grows.